DISTRIBUTED AI SYSTEMS USING PYSPARK RDDs – EXAM ANSWERS

Below are the **5 scenarios in the same format as before**, suitable for **10 marks each**, with **objective, architecture, working, PySpark/RDD concepts, intelligent agents, advantages, and conclusion**.

---

1. HYBRID AI RECOMMENDATION ENGINE DESIGN

10 Marks

Introduction

A **Hybrid Recommendation System** combines **Content-Based Filtering** and **Collaborative Filtering** to generate more accurate and personalized recommendations.

PySpark RDDs can distribute large-scale user-item interaction data across multiple machines, while intelligent agents continuously adapt recommendations based on user behaviour and feedback.

Objective

The system aims to:

- Process millions of user-item interactions.
- Combine content-based and collaborative filtering.
- Generate personalized recommendations.
- Adapt recommendations in real time.
- Handle large datasets using distributed processing.

System Architecture

Users

↓

User-Item Interactions

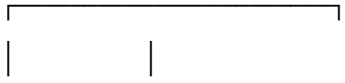
↓

Distributed Data Ingestion

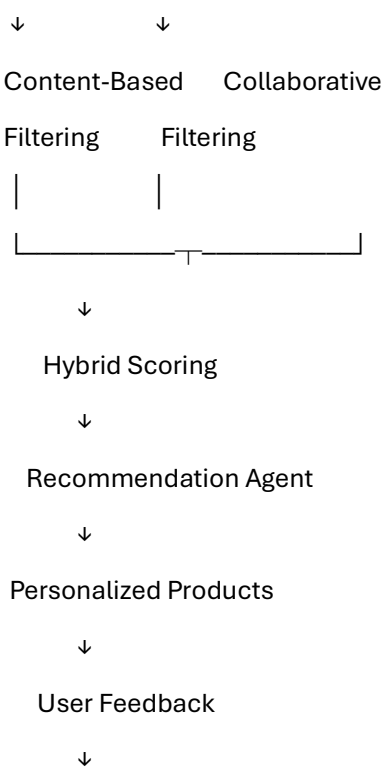
↓

PySpark RDD Processing

↓



CO5 AT1



Content-Based Filtering

This approach recommends items similar to those the user previously liked.

Example:

User likes:

Action + Sci-Fi Movies

Recommend:

Other Action + Sci-Fi Movies

Features may include:

- Product category.
- Genre.
- Brand.
- Keywords.

- Product description.

Collaborative Filtering

Collaborative filtering uses the behaviour of similar users.

Example:

User A likes → Laptop, Mouse, Keyboard

User B likes → Laptop, Mouse

Therefore:

Recommend Keyboard to User B

PySpark RDD Processing

Suppose the dataset contains:

(user_id, item_id, rating)

RDD operations can process the data in parallel.

```
ratings = sc.textFile("ratings.csv")

user_items = ratings.map(
  lambda x: x.split(",")
)

grouped = user_items.groupBy(
  lambda x: x[0]
)

recommendations = grouped.mapValues(
  lambda x: list(x)
)
```

Important RDD Transformations

- map()

- filter()
- flatMap()
- reduceByKey()
- groupByKey()
- join()

These operations allow large-scale user-item data to be processed across distributed nodes.

Hybrid Recommendation

The two recommendation scores can be combined:

[
Score=\alpha C+(1-\alpha)CF
]

where:

- (C) = Content-Based score.
- (CF) = Collaborative Filtering score.
- (\alpha) = weighting parameter.

Role of Intelligent Agent

The **Recommendation Agent**:

1. Observes user behaviour.
2. Collects clicks, ratings and purchases.
3. Updates user preferences.
4. Generates recommendations.
5. Learns from feedback.

Example:

User clicks Sports Shoes

↓

Agent detects interest

↓

Update User Profile

↓

Recommend Running Shoes

CO5 AT1

↓

User Feedback

↓

Update Recommendation

Advantages

- More accurate recommendations.
- Handles large datasets.
- Reduces cold-start problems.
- Supports real-time adaptation.
- Scalable using PySpark.

Conclusion

A Hybrid Recommendation Engine combines the strengths of content-based and collaborative filtering. PySpark RDDs provide scalable distributed processing, while intelligent agents continuously adapt recommendations according to user behaviour and feedback.

---

2. SMART DISASTER DETECTION SYSTEM

10 Marks

Introduction

A **Smart Disaster Detection System** uses AI, distributed processing and multiple data sources to detect natural disasters such as floods, earthquakes and landslides.

PySpark RDDs process large volumes of sensor, satellite and social-media data, while intelligent agents coordinate detection and emergency response.

Objective

The system aims to:

- Detect disasters early.
- Combine multiple data sources.
- Process streaming information.
- Generate early warnings.
- Coordinate emergency responses.

CO5 AT1

System Architecture

Data Sources

1. Satellite Data

Can provide:

- Flooded areas.
- Changes in land surface.
- Cloud patterns.
- Vegetation changes.

2. Sensor Data

Examples:

- Water level.
- Temperature.
- Seismic activity.
- Atmospheric pressure.

3. Social Media

Posts can contain information about:

- Flood locations.
- Earthquake impacts.
- Road blockages.
- People requiring assistance.

PySpark RDD Processing

Each data source can be represented as an RDD.

```
sensor_rdd = sc.textFile("sensor_data.csv")
social_rdd = sc.textFile("social_media.csv")
satellite_rdd = sc.textFile("satellite_data.csv")
```

The data can then be filtered and transformed:

```
high_water = sensor_rdd.filter(
  lambda x: float(x.split(",")[2]) > 8
)
```

Data Fusion

Data fusion combines information from multiple sources.

Example:

Sensor:

High Water Level

+

Satellite:

Water-covered Area

+

CO5 AT1

Social Media:

"Flood near Chennai"

↓

High Confidence Flood Detection

This is more reliable than depending on a single source.

Intelligent Agents

Sensor Agent

Collects and monitors sensor information.

Detection Agent

Analyses data and identifies possible disasters.

Decision Agent

Determines severity and response requirements.

Communication Agent

Sends warnings to authorities and citizens.

Early Warning

Abnormal Sensor Data

↓

Detection Agent

↓

AI Risk Analysis

↓

High Risk

↓

Communication Agent

↓

Emergency Alert

Advantages

- Early disaster detection.

- Multi-source data fusion.
- Scalable processing.
- Real-time decision-making.
- Improved emergency coordination.

Conclusion

A distributed disaster detection system combines satellite, sensor and social-media information using PySpark. Intelligent agents coordinate detection, warning generation and emergency response, allowing faster and more reliable disaster management.

---

3. AI-BASED FRAUD DETECTION FRAMEWORK

10 Marks

Introduction

An **AI-based Fraud Detection System** identifies suspicious financial transactions by analysing large-scale transaction data.

PySpark RDDs distribute transaction processing, while intelligent agents detect anomalies, generate alerts and learn from confirmed fraud cases.

Objective

The system aims to:

- Process millions of transactions.
- Detect abnormal behaviour.
- Identify suspicious patterns.
- Generate real-time alerts.
- Continuously improve detection accuracy.

System Architecture

Financial Transactions

↓

Data Ingestion

↓

PySpark RDD

↓

CO5 AT1

Data Preprocessing

↓

Feature Extraction

↓

Distributed AI Model

↓

Anomaly Detection

↓

Fraud Detection Agent

↓

Risk Score

↓

Alert System

Transaction Features

The system can analyse:

- Transaction amount.
- Transaction frequency.
- Location.
- Time.
- Merchant.
- Device.
- Previous transaction behaviour.

RDD Processing

Transaction data can be loaded into an RDD:

```
transactions = sc.textFile(
  "transactions.csv"
)


large_transactions = transactions.filter(
```

```
  lambda x: float(x.split(",")[2]) > 100000
)
```

Transactions can be grouped by customer:

```
customer_transactions = transactions.map(
  lambda x: (x.split(",")[0], x)
).groupByKey()
```

Anomaly Detection

A transaction can be considered suspicious if it significantly differs from normal behaviour.

Example:

Normal:

₹2,000 → Chennai → Regular Device

Suspicious:

₹2,00,000 → Foreign Location → Unknown Device

The system assigns a risk score:

$$
[
Risk=w_1A+w_2L+w_3F+w_4D
]
$$

where:

- (A) = amount anomaly.
- (L) = location anomaly.
- (F) = frequency anomaly.
- (D) = device anomaly.

Intelligent Agents

Monitoring Agent

Continuously monitors transactions.

Fraud Detection Agent

Identifies suspicious patterns.

Alert Agent

Generates warnings.

CO5 AT1

Learning Agent

Uses confirmed fraud cases to improve future detection.

Agent Collaboration

Transaction

↓

Monitoring Agent

↓

Fraud Detection Agent

↓

Risk Score

↓

┌──────────┐

↓ ↓

Low Risk High Risk

↓ ↓

Allow Alert

↓

Investigation

Continuous Improvement

When investigators confirm fraud:

Confirmed Fraud

↓

Learning Agent

↓

Update Fraud Patterns

↓

Improve Future Detection

Advantages

- Scalable.

- Fast transaction analysis.
- Detects unusual behaviour.
- Supports real-time alerts.
- Continuously improves.

Conclusion

A PySpark-based fraud detection framework can process large transaction datasets efficiently. Intelligent agents can collaborate to identify anomalies, generate alerts and learn from newly confirmed fraud patterns.

---

4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

10 Marks

Introduction

An **Autonomous Supply Chain System** uses distributed AI and intelligent agents to optimise inventory, transportation and demand forecasting.

PySpark processes large-scale supply-chain data, while multiple agents make decentralised decisions.

Objective

The system aims to:

- Reduce inventory costs.
- Improve demand forecasting.
- Optimise transportation.
- Prevent stock-outs.
- Improve supply-chain efficiency.

System Architecture

Sales Data

↓

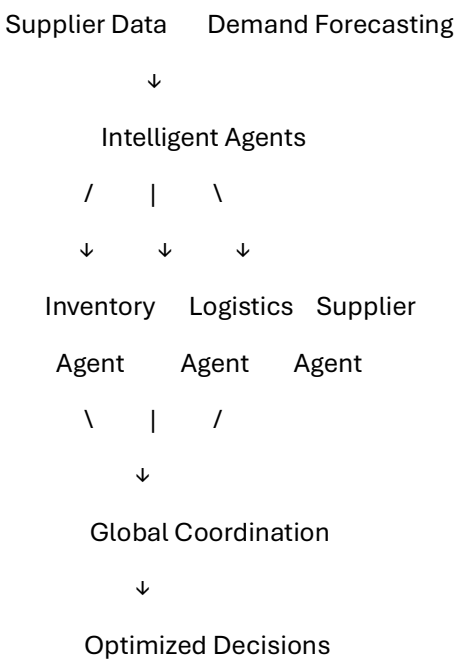
Inventory Data —→ PySpark Processing

↓ ↓

Logistics Data RDD Analysis

↓ ↓

CO5 AT1



RDD Transformations

Supply-chain data can be processed using:

map()

Transforms each record.

filter()

Removes invalid or irrelevant records.

reduceByKey()

Aggregates values by key.

Example:

```
sales = sc.textFile("sales.csv")


product_sales = sales.map(
  lambda x: (
    x.split(",")[1],
    int(x.split(",")[2])
  )
)
```

CO5 AT1

```
total_sales = product_sales.reduceByKey(
   lambda a, b: a + b
)
```

Intelligent Agents

Inventory Agent

Monitors stock levels.

Demand Forecasting Agent

Predicts future demand.

Logistics Agent

Optimises transportation routes.

Supplier Agent

Monitors supplier availability and delivery times.

Agent Interaction

Demand Agent

↓

High Future Demand

↓

Inventory Agent

↓

Increase Stock

↓

Logistics Agent

↓

Select Efficient Route

↓

Supplier Agent

↓

Order Raw Materials

CO5 AT1

Cost Minimisation

The system tries to minimise:

[
Total\ Cost=
Inventory\ Cost+
Transportation\ Cost+
Ordering\ Cost
]

Agents coordinate their decisions to reduce the overall cost.

Example

Suppose a product's predicted demand increases.

Demand ↑

↓

Forecasting Agent

↓

Inventory Agent → Increase Stock

↓

Supplier Agent → Place Order

↓

Logistics Agent → Optimise Delivery

This prevents stock-outs while controlling transportation and inventory costs.

Advantages

- Reduced operational cost.
- Better inventory management.
- Faster decision-making.
- Distributed processing.
- Improved demand forecasting.
- Autonomous coordination.

Conclusion

An autonomous supply-chain system combines PySpark's distributed data processing with multiple intelligent agents. Each agent manages a specialised task while coordinating with other agents to improve overall efficiency and minimise costs.

---

5. PERSONALIZED LEARNING ANALYTICS PLATFORM

10 Marks

Introduction

A **Personalized Learning Analytics Platform** uses AI to analyse student behaviour and automatically adapt educational content.

PySpark RDDs enable large-scale processing of student interaction data, while intelligent learning agents provide personalised recommendations.

Objective

The system aims to:

- Analyse student behaviour.
- Identify weak areas.
- Track academic performance.
- Adapt content difficulty.
- Provide real-time feedback.
- Support thousands of learners.

System Architecture

Student Interactions

↓

Quiz / Video / Assignment Data

↓

Data Ingestion

↓

PySpark RDDs

↓

Learning Analytics

↓

CO5 AT1

Student Profile

↓

Learning Agent

↓

Personalized Content

↓

Student

↓

Feedback

↓

Agent Learning

Data Sources

The platform can collect:

- Quiz scores.
- Assignment marks.
- Video watching time.
- Number of attempts.
- Question-solving time.
- Login frequency.
- Topic completion.

RDD Processing

```
student_data = sc.textFile(
  "student_data.csv"
)

scores = student_data.map(
  lambda x: (
    x.split(",")[0],
    float(x.split(",")[2])
```

```
  )
)

average_scores = scores.groupByKey().mapValues(
  lambda values: sum(values) / len(values)
)
```

The RDD allows student records to be processed across distributed computing nodes.

Learning Analytics

The system can calculate:

$$
\begin{aligned}
&[\\
&\text{Performance} =\\
&w\_1\text{Score}+w\_2\text{Completion}+w\_3\text{Accuracy}\\
&]
\end{aligned}
$$

Based on the performance, the agent determines the student's learning level.

Example:

Score > 80%

↓

Advanced Content

Score 50–80%

↓

Intermediate Content

Score < 50%

↓

Basic Content + Revision

Intelligent Learning Agent

The agent:

1. Observes student activity.
2. Analyses performance.

3. Identifies weak topics.
4. Selects suitable content.
5. Receives feedback.
6. Updates the student learning profile.

Real-Time Feedback

Student completes quiz

↓

Score calculated

↓

PySpark Analytics

↓

Learning Agent

↓

Weak Topic Detected

↓

Revision Material

↓

New Quiz

Scalability

If thousands of students use the platform simultaneously:

Thousands of Students

↓

Large Interaction Dataset

↓

PySpark Cluster

/ | \

Worker Worker Worker

\ | /

↓

CO5 AT1

Learning Analytics

↓

Personalized Recommendations

RDDs distribute the workload across multiple worker nodes.

Advantages

- Personalised learning.
- Real-time feedback.
- Scalable to thousands of students.
- Identifies weak topics.
- Adapts content difficulty.
- Supports data-driven education.

Conclusion

A Personalized Learning Analytics Platform combines PySpark RDDs with intelligent learning agents to process large amounts of student data. The system continuously analyses performance and adapts learning content, making education more personalised and scalable.

QUICK COMPARISON

Scenario	Main AI System	PySpark/RDD Role	Intelligent Agent Role
1. Recommendation Engine	Hybrid AI	Process user-item data	Adapt recommendations
2. Disaster Detection	Distributed AI	Process multi-source data	Detect, coordinate and warn
3. Fraud Detection	Anomaly Detection	Process transactions	Detect and alert
4. Supply Chain	Optimization AI	Process sales/logistics data	Decentralized decisions
5. Learning Analytics	Adaptive AI	Process student data	Personalize learning

One-Line Memory Trick

Recommendation → "What should I recommend?"

Disaster → "What is happening?"

Fraud → "Is this transaction suspicious?"

CO5 AT1

Supply Chain → "How can I reduce cost?"

Education → "How can I help this student learn better?"

Common PySpark RDD flow for all five:

Large-Scale Data

↓

RDD

↓

map / filter / reduceByKey / join

↓

Distributed Processing

↓

AI Model

↓

Intelligent Agent

↓

Real-Time / Optimized Decision